



Learning to trade in financial time series using high-frequency through wavelet transformation and deep reinforcement learning

Jimin Lee¹ · Hayeong Koh² · Hi Jun Choe²

Accepted: 14 January 2021

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC part of Springer Nature 2021

Abstract

Deep learning-based financial approaches have received attention from both investors and researchers. This study demonstrates how to optimize portfolios, asset allocation, and trading systems based on deep reinforcement learning using three frameworks. In the proposed deep learning structure, the input data are first decomposed through wavelet transformation (WT) to remove noise from stock price time-series data. Then, only the mother wavelet (high-frequency) data are used as input. Second, reinforcement learning is performed using the high-frequency data. The reinforcement learning network employs long short-term memory (LSTM). Actions are determined by the LSTM network or randomly. Third, it learns the optimal investment trading system using the actions of a given transaction and appropriate rewards. The structure of the optimal investment trading system obtained by the proposed deep reinforcement learning structure improves trading performance without requiring the construction of a predictive model. To investigate the performance of the proposed structure, we applied the S&P500, DJI, and KOSPI200 indices to the proposed structure (HW_LSTM_RL) and other reinforcement learning structures for comparison. We evaluated the difference in Sharpe ratio for various test periods (one to three years) and for different rewards. Using the decomposed high-frequency data as input, a portfolio of investment transactions was improved for highly volatile markets. In deep reinforcement learning, we found that network composition and appropriate rewards have significant influence on learning transactions in financial time-series data. Thus, the proposed HW_LSTM_RL structure demonstrates the importance of input data composition, learning network settings, and rewards.

Keywords Deep reinforcement learning (Deep RL) · Wavelet transforms · LSTM · Wavelet deep LSTM Q—network · High frequency · Trading system · Portfolio optimization · Sharpe Ratio

1 Introduction

Over the past decade, a variety of supervised learning methods have been employed to predict future returns of financial assets using machine learning techniques. For example, supervised learning has been used to predict stock prices

by calculating the probability of increases and decreases in stock prices. If a stock price is predicted to increase, the investment trade should be to buy, and, if the stock price is predicted to decline, the stock should be sold. However, making an investment decision based on a given forecast does not mean a successful investment transaction and does not maximize risk-free returns. In other words, risk-free returns may or may not be maximized. Then, we began to consider the best way to learn successful investment trading behaviors, and this is the motivation of the current study. The propose to use reinforcement learning to determine an action that can maximize the cumulative reward for the action according to the state, where the reward is compensated according to the return and loss of the initial investment cost according to the change in the portfolio value. In addition, we consider how to learn the transaction itself using deep reinforcement learning.

Deep reinforcement learning can include multiple factors for transaction costs and environment, and can consider

✉ Jimin Lee
jmlee524@ewha.ac.kr; jimin.lee524@gmail.com

Hayeong Koh
hykoh@yonsei.ac.kr

Hi Jun Choe
choe@yonsei.ac.kr

¹ Institute of Mathematical Sciences, Ewha Womans University, Seoul, 03760, Republic of Korea

² Department of Mathematics Yonsei University, 50 Yonsei-Ro, Seodaemun-Gu, Seoul, 03722, Republic of Korea

buying, selling, and holding behaviors as stock information. Unlike machine learning, where an action or a set of actions is selected, here, the goal is to maximize the reward, and the predicted value is a policy or execution strategy. Therefore, it is important to implement an optimized design, e.g., states, reward, environment, and action, depending on the given situation. Agents can make it like a real stock market transaction by adding various constraints, e.g., transaction costs. Here, the stock price (i.e., preprocessed input data) represents the states, and the actions according to the states are buy, sell, and hold. In the environment, taxes and the like can be set, and rewards are set using only the rate of change in the portfolio value. Here, the ultimate goal is to employ deep reinforcement learning to optimize a portfolio based on appropriate numerical rewards for a given action. It also learns actual data (Standard and Poor's (S&P500), Dow Jones Industrial Average Index (DJI), and KOSPI200 indices) used in the market to ensure data reliability. Unlike many other models that generally seek to describe stock time-series data, the proposed structure can describe stock investment transactions without using an appropriate model to describe investment transactions. In other words, using deep reinforcement learning, we can learn investment transactions without a model for time-series data analysis. Here, the input data are preprocessed to learn the investment transaction of deep reinforcement learning more effectively. In addition, noise in the time-series data is removed, and wavelet transform (WT) is applied to decompose both high-frequency and low-frequency data.

WT is used rather than Fourier transform for noise removal because Fourier transform only provides information about the frequency spectrum, i.e., it does not provide any information about when an event occurs. In other words, it is difficult to properly analyze a signal whose frequency changes each time. For example, Fourier transform cannot distinguish between signal 1 that is 10Hz for 0 to 10 seconds and 20Hz for 10 to 20 seconds and signal 2 where 10Hz and 20Hz coexist. Here, with Fourier transform, only 10Hz and 20Hz can be reported for 0 to 20 seconds. This is as if the notes in the score are not arranged in time, but rather according to the pitch (frequency) of the signal. One method proposed to overcome these limitations is the short time Fourier transform (STFT), which divides a long time-varying signal into shorter time units, and then applies Fourier transform. As a result, we can observe the frequencies present in each time interval. As the signal is divided into short time units, it is better to know which frequencies exist at which time, and, if the signal is divided into relatively long time units, it is better to know which frequency exists in the given time. Thus, the temporal resolution improves as the width of the window dividing the signal becomes smaller. In contrast, the frequency resolution

improves as the window width increases. Good resolution means we can analyze it in more detail. However, for short-time Fourier transform, it is impossible to improve the frequency and time resolutions simultaneously; however, WT increases the temporal and frequency resolutions of the high-frequency component signals. For low-frequency signals, the frequency resolution increases, and the time resolution decreases. As Mehala & and Dahiya [26] stated, if STFT must abandon the time or frequency resolution, the WT is more suitable than Fourier transform for time and frequency decomposition of time-series data because WT can consider both the time and frequency resolutions. Basic and mathematical literature on wavelets has been published by Addison, Lahmiri, and Walnut [1, 21, 41].

Thus, prior to employing deep reinforcement learning, we preprocess the data via WT. Data decomposed by WT is decomposed into high-frequency (mother wavelets) and low-frequency (father wavelets) components. Here, high frequencies describe the details of time-series data, and low frequencies represent a smoothly represented full waveform. In addition, Zhang et al. and Renaud et al. [33, 48] reported that wavelets improve financial time-series forecasting. However, we seek obtain profit even if the stock price increase, and we want to obtain the least damage or profit even if the stock price is reduced. Therefore, by using the discrete WT proposed by Mallat [25], the time-series data are decomposed into high- and low-frequency data. Only the high-frequency data are used for training to ensure stable and high profitability as Sherstov & Stone, Hsieh et al., and Lahmiri [17, 21, 36]. Low-frequency signals are helpful in identifying overall trends or waveforms; however, low-frequency signals were not used because they are not suitable if detailed information is required. Among the many wavelets, the Haar wavelet was selected because it can better describe stock investment transactions according to Stanković & Falkowski [38]. According to Choe et al. [9], instant jumps occur as soon as our buy or sell stock. We use Haar wavelets because the shape of the instantaneous tic-toc jump is similar to the shape of the tic-toc as the level of the Haar wavelet increases. It was determined that using high-frequency data decomposed by Haar wavelets is suitable for investment transactions. Similarly, Sherstov & Stone [36] studied high-frequency reinforcement learning. Reinforcement learning can be used to build a trading system for daily FX data. Li et al. [22] investigated a hybrid predictive model by combining genetic programming and wavelet decomposition to financially process the various decomposition measures of the original stock index. In addition, in a recent study by Qiu et al. [32], wavelet transform based on LSTM and attention mechanisms was used to denoise historical stock data, extract and train its functions, and develop a stock price prediction model. This study was

compared to three other models, including the GRU neural network model.

Deep reinforcement learning comprises three important frameworks for optimizing our trading system. First, input data are decomposed through WT to remove noise from stock price time-series data, and then only high-frequency data are used as the input data. Second, reinforcement learning is performed on the input data with noise removed. Here, the learning network for reinforcement learning uses LSTM. Depending on the learning method of reinforcement learning, an action is determined by the LSTM network or randomly. Third, we update the deep reinforcement learning network to prove the transaction outcome by changing the state according to the following circumstances. We also compare the difference between the behavior of a given transaction and the appropriate reward. In addition, the differences between the proposed HW_LSTM_RL structure and similar structures are confirmed through the Sharpe ratio. By comparing Sharpe ratio as done by Moody et al., Moody & Saffell, and Xiong [28, 29, 47], as well as the daily and cumulative returns, we confirmed that the proposed structure yields the best results.

Our primary contributions are summarized as follows.

1. Through the proposed HW_LSTM_RL structure, we confirm that the performance of learning financial time series-transactions is improved.
2. High-frequency input has important results for learning financial data. The high-frequency data provide useful information for learning investment trading and describe market volatility in detail. Therefore, the proposed HW_LSTM_RL structure is suitable for investment transactions in highly volatile markets, e.g., financial market diversification.
3. In deep reinforcement learning, proper configuration of the learning network and appropriate reward settings according to the given environment improve learning outcomes.

The remainder of this paper is organized as follows. In Section 2, we introduce recent studies that used reinforcement learning and wavelet transformation for stock price learning. Section 3 describes WT, optimal deep reinforcement learning, and the Sharpe ratio used in our experiments. Section 4 describes the experimental settings for reinforcement learning states, actions, environment, and rewards, and Section 5 compares the experimental results obtained by the proposed HW_LSTM_RL with other similar structures. In addition, the differences in Sharpe ratio, daily return, and cumulative return according to the rewards are evaluated. Sections 6 and 7 research are introduced in conclusions and Future works.

2 Related work

Applying reinforcement learning to financial asset trading began with Neuneier Neuneier [31], who applied portfolio management to Markovian decision-making problems, and the outcomes of that study went beyond heuristic and supervised learning. Then, they recurrent reinforcement learning (RRL) was studied by Moody et al. and Moody & Saffell [28, 29]. This study had significant impact on RRL research. They learned portfolio management of risky and non-risked assets through repeated lectures and suggested differential Sharpe ratio calculations. In addition, Gold [15] and Bertoluzzo & Corazza [4] used RRL to consider the complexity of a decision function as a neural network and added a hidden layer to learn investment transactions. Dempster & Leemans [10] performed a risk-focused study, and Deng et al. [12] investigated how to improve trading system performance by introducing sparse coding into RRL. Deng [11] conducted another study on deeper iterative reinforcement learning with higher level feature extraction, and they extended backpropagation over time to train the agent efficiently. Carapuço [7] described a system for short-term speculation in the foreign exchange market based on reinforcement learning (RL). Meng & Khushi [27] used reinforcement learning to review all recent stock/foreign forecasts or trades.

Recently, research using deep reinforcement learning for stock investment transactions has been conducted. For example, Hu & Lin [18] studied the main research issues in policy optimization for financial portfolio management. In addition, Xiong [47] selected 30 stocks as trading stocks, utilized daily prices for training and trading market environments, a deep reinforcement learning agent training and adaptive trading strategies to evaluate agent performance. Li et al., Li. et al., Brim, and Gao et al. [6, 14, 23, 24] presented deep reinforcement learning theory in stock trading decision making and stock price predictions. Fengqian & Chao [13] puts the learning features clustered by financial data K-line theory into the model, and through deep reinforcement learning method to realize online adaptive control of parameters in the unknown environment to realize high-frequency trading strategies. Carta et al. [8] presented multi-layer and multi-ensemble stock traders to validate the effectiveness of the approach in real-world trading scenarios. Wu et al. [46] used the GRU to extract information functions from raw financial data and technical indicators. In addition, this paper proposed two trading strategies, GDQN (Gated Deep Q-learning trading strategy) and GDPG (Gated Deterministic Policy Gradient trading strategy), as internal reinforcement learning methods for custom design of state and action spaces. This strategy achieved more stable returns than the state-of-the-art DRL trading strategy in a volatile market. In addition, Soleymani & Paquet [37] studied the DeepBreath

method, which is based on deep reinforcement learning and combines limited stacked auto-encoders and convolutional neural networks to keep only the most informative abstract functions. This study presents an investment strategy that minimizes market risk using blockchain to address settlement risks due to delays between acquisition and payment of assets that fail to fulfill the terms of the contract.

3 Methodology

This section The first briefly describes the background of the methodology used in the proposed structure, and then presents the proposed structural solution.

3.1 Wavelet transform

Wavelet transform can better explain the time and frequency resolutions of high-frequency component signals compared to Fourier transform, which only provides information about the frequency spectrum and not the time of event occurrence. Formally, the wavelet series represents a squared-integral function based on a complete set of orthogonal primitives called *wavelets*, i.e., a small wave. There are two ways analysis can be extended relative to Fourier transform. First, in WT, the basis functions (wavelets) are more complex than the sine and cosine applied in Fourier transform. Second, analysis is performed at multiple scales.

Wavelets provide a certain degree of localization in time (space). However, due to the uncertainty principle of Wiener Heisenberg, total spatial frequency localization remains impossible. Sharp spikes and discontinuities typically use fewer wavelet criteria than sine-cosine basis functions in Fourier transforms. In addition, localization in the spatial domain with wavelet localization in frequency yields a sparse representation of many real signals (images). Scarcity opens up successful applications, e.g., data or image compression, noise filtering, and image minutia detection.

The wavelets can be characterized by mother and daughter wavelets. A *Mother wavelet* Ψ is a wavelet function that characterizes the shape and covers the entire domain of interest. All other derived wavelets are referred to as *daughter wavelets*, which are defined in terms of the parent wavelet. Here, the generating (basis) function is $\Psi_{s,\tau}(x)$, where s characterizes the scale of a wavelet function, and τ characterize the shifts of a wavelet function.

For continuous wavelet transform(CWT), the wavelets are generated from the single mother wavelet $\Psi(t)$ by scaling s and shifting τ ; $s > 1$ dilates, $s < 1$ contracts the signal, the daughter wavelet can be defined by

$$\Psi_{s,\tau}(t) = \frac{1}{\sqrt{s}} \Psi\left(\frac{t-\tau}{s}\right), \quad (1)$$

where Ψ is the basis wavelet, the coefficient $\frac{1}{\sqrt{s}}$ is used because the energy has to be normalized across different scales.

Let $f(t)$ denote a square-integrable function ($f(t) \in L^2(\mathbb{R})$); then CWT with the wavelet Ψ can be defined as follows :

$$c(s, \tau) = \int_{\mathbb{R}} f(t) \Psi_{s,\tau}^*(t) dt, \quad (2)$$

where $s \in \mathbb{R}^+ - \{0\}$ is scale factor, $\tau \in \mathbb{R}$ is shift factor (translation factor), $c(s, \tau)$ are wavelet coefficients, and the complex conjugation is denoted by $*$. This inner product tells us how the signal looks like (correlated) to the local probe given by a particular wavelet. The inverse transform of the continuous wavelet transform, the finite energy signal $f(t)$ from the wavelet coefficient $c(s, \tau)$ is represented by

$$f(t) = \int_{\mathbb{R}^+} \int_{\mathbb{R}} c(s, \tau) \Psi_{s,\tau}(t) ds d\tau. \quad (3)$$

Wavelet transformations are generally defined without the need to specify a specific mother wavelet Ψ . The users can choose or design a mother wavelet Ψ based on application needs. The mother wavelet is used to create generating (basis) functions $\Psi_{s,\tau}(t)$ used in the above expansion. The coefficient $c(s, \tau)$ can be interpreted as being similar to the frequency spectrum (spectrogram) in the Fourier transform.

Since the coefficients of the continuous wavelet transform have a significant amount of redundant information, it is necessary to sample the coefficients to reduce the redundancy. Disassembling the time series into orthogonal sets results in a discrete wavelet transform (DWT). In Mallat [25], they proposed a discrete wavelet transform implementation that uses a pair of high-pass filters and low-pass filters to filter time series. There are two types of Mallat algorithms: the father wavelet $\Phi(t)$ and the mother wavelet $\Psi(t)$. In other words, the mother wavelets describe high-frequency (detail) components and the father wavelets represent low-frequency (smooth) components of time series. The father wavelet $\Phi(t)$ and the mother wavelet $\Psi(t)$ are integrated into 1 and 0, respectively, and can be formulated as follows as Mallat [25] :

$$\int \Phi(t) dt = 1, \int \Psi(t) dt = 0. \quad (4)$$

The father wavelets Φ and the mother wavelets Ψ are expressed by j and k ,

$$\begin{aligned} \Phi_{j,k}(t) &= 2^{-\frac{j}{2}} \Phi(2^{-j}t - k), \\ \Psi_{j,k}(t) &= 2^{-\frac{j}{2}} \Psi(2^{-j}t - k) \end{aligned} \quad (5)$$

where $k = 0, 1, 2, \dots$, and $j = 0, 1, 2, \dots, J$, where J denote the number of multi-resolution scales.

Any function $f(t)$ can be decomposed by a sequence of projections onto wavelet basis as follows in Mallat [25]:

$$\begin{aligned} a_{J,k} &= \int f(t) \Phi_{J,k}(t) dt, \\ d_{j,k} &= \int f(t) \Psi_{j,k}(t) dt, \end{aligned} \quad (6)$$

where $a_{J,k}$ and $d_{j,k}$ are signal representation in wavelet domain. $a_{J,k}$ is smooth coefficients, and $d_{j,k}$ is detail coefficients.

The signal decomposition of time series using DWT is shown in Fig. 1 according to the principle of convolution. When decomposing the original signal $f(t)$, two components A_1 and D_1 are generated by combining the signals into the decomposition low-pass filter D_L and decomposition high-pass filter D_H , respectively. A_1 is decomposed into D_2 and A_2 on a larger scale. Also, A_2 is decomposed into D_3 and A_3 on a larger scale and so on. A_1 is an approximate signal showing the general trend of low frequency components. D_1 is the detail of the signal and is related to the high frequency component of the signal. In terms of components, the orthogonal wavelet series approximation can be expressed by as follows in Mallat [25]:

$$\begin{aligned} f(t) &= A_J(t) + \sum_{i=1}^J D_i \\ &= A_J(t) + D_J(t) + D_{J-1}(t) + \cdots + D_1(t) \end{aligned} \quad (7)$$

where $j = 0, 1, 2, \dots, J$, and $k = 0, 1, 2, \dots$, $A_J(t) = \sum_k a_{J,k} \Phi_{J,k}(t)$, and $F D_j(t) = \sum_k d_{j,k} \Psi_{j,k}(t)$. The DWT is used to calculate the coefficient of the wavelet series approximation in (7) for a discrete signal $f_1, f_2, \dots, f_n$ with finite extent. $A_J(t)$ is the coarser approximation of the input time series $f(t)$. If the financial time series is very rough, we can iteratively apply discrete wavelet transforms to reduce the excessive risk. As a result, two-level wavelets

are applied twice for data preprocessing in this study, as proposed in Bao et al. and Hsieh et al. [2, 17].

There are various types of wavelets, but we use Haar wavelet. Haar wavelets are useful for time series analysis because they can capture fluctuations between adjacent observations in Lahmiri [21]. In addition, according to the paper Choe [9], the stock time series data is similar to Haar's appearance because the instantaneous jumps in trading moments of buying and selling appear. Therefore, Haar wavelet is used. Haar's mother wavelet and father wavelet use the following formula:

$$\Phi(t) = \begin{cases} 1 & 0 \leq t \leq 1, \\ 0 & \text{otherwise.} \end{cases} \quad (8)$$

$$\Psi(t) = \begin{cases} 1 & 0 \leq t \leq \frac{1}{2}, \\ -1 & \frac{1}{2} \leq t \leq 1. \end{cases} \quad (9)$$

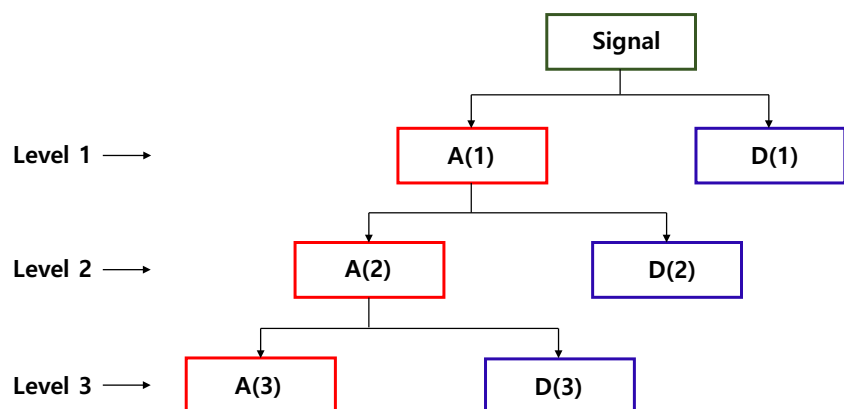
DWT handles very irregular time series data series flexibly. In our deep reinforcement learning architecture, only the high frequency wavelets are used as data input among the high frequency and low frequency resolved wavelets.

3.2 LSTM, the reinforcement learning network

In the process of learning investment trading through reinforcement learning, we use a policy to determine an action using the network or by random behavior. Here, the network that determines the policy uses the LSTM proposed by Hochreiter & Schmidhuber [16]. In other words, in the reinforcement learning process, after determining the action by LSTM network, we are rewarded for that action. In addition, the learning network is updated to maximize the cumulative reward. The structure of reinforcement learning is detailed in Section 3.3. This section provides a brief overview of the LSTM structure.

If the action is determined by learning (rather than random determination), we use LSTM. Time-series data or

Fig. 1 Decomposition of signal using DWT



sequential data can be found in several studies that used a recurrent neural network (RNN) or LSTM to improve the outcomes of the results in Hochreiter & Schmidhuber, Krizhevsky et al., Sundermeyer et al., and Bao et al. [2, 16, 20, 39].

Theoretically, RNNs can solve long-term dependency problems; however, the larger the time gap to obtain the required information, the more difficult it is for RNNs to keep up with the information it learns. The reasons why RNNs could not solve the long-term dependency problem were revealed by Begio et al. and Hochreiter & Schmidhuber [3, 16]. The reason is that when RNN backpropagation, the gradient becomes too small for learning. During RNN backpropagation, $\tanh$'s gradient is multiplied; thus, $\tanh$ may have some effect on the gradient vanishing problem. LSTM complements the long-term dependency problem in RNN.

The core idea of the LSTM is the cell state, it likes a conveyor belt, continually driving the entire chain and applying only a small linear interaction. Here, the information does not change at all. LSTM can add or remove something from the cell state, which is controlled by a structure called a *gate*. The gate is an additional way that information can be conveyed, comprising a sigmoid layer and a pointwise product. The LSTM formula corresponding is given as follows Hochreiter & Schmidhuber [16] :

$$\begin{aligned}
 f_t &= \sigma(\mathbf{w}_f \cdot [h_{t-1}, x_t] + b_f), \\
 i_t &= \sigma(\mathbf{w}_i \cdot [h_{t-1}, x_t] + b_i), \\
 \tilde{C}_t &= \tanh(\mathbf{w}_C \cdot [h_{t-1}, x_t] + b_C), \\
 C_t &= f_t \times C_{t-1} + i_t \times \tilde{C}_t, \\
 h_t &= o_t \times \tanh(C_t), \\
 o_t &= \sigma(\mathbf{w}_o \cdot [h_{t-1}, x_t] + b_o),
 \end{aligned} \tag{10}$$

where f_t is the value of the forget gate layer, i_t is the value of the input gate layer, o_t is the value of the output gate layer, $\mathbf{w}_f$ are forget weights, $\mathbf{w}_i$ are input weights, $\mathbf{w}_o$ are output weights, b_f is the forget gate bias, b_i is the input gate bias, b_o is the output gate bias, and $\sigma = \frac{1}{1+e^{-x}}$ is sigmoid function. The sigmoid function refers to a special form of a logistic function that takes the form of an S curve. The sigmoid function determines whether to remember or forget this value.

The first step in LSTM is to determine what information is discarded from the cell state, which is determined by the sigmoid layer. The gate of this stage is referred to as the forget gate layer. This step takes h_{t-1} and x_t , and sends a value between 0 and 1 to cell state C_{t-1} . Here, a value of 1 means preserve all information, and a value of 0 means

discard information. The next step is to determine which new information comes into the cell state. First, the value the sigmoid layer will update, referred to as the input gate layer, is determined. Then, the $\tanh$ layer creates a new candidate vector $\tilde{C}_t$ and prepares it for the cell state. As a result, the information from the two steps is combined to update the state. Then, the old state C_{t-1} updated to create a new cell state C_t . Here, we have already determined the values to update in the previous step. First, we multiply the previous state by f_t and really forget what we decided to forget in the first step. Then, $i_t \times \tilde{C}_t$. This addition value is scaled by how much to update the value decided to update in the second step. Finally, we must specify what to export to the output. This output is filtered based on the cell state.

3.3 Proposed structure

Our research demonstrates how to optimize a portfolio, asset allocation, and trading system without requiring the construction of a deep learning-based model. As shown in Fig. 2, the reinforcement learning framework comprises three steps that are important for optimizing the trading system. The learning process of the proposed HW_LSTM_RL structure is presented in Algorithm 1 through simple pseudocode.

The framework of the proposed HW_LSTM_RL structure is described as follows. First, input data are decomposed via WT to remove noise in the stock price time-series data. Here, only mother wavelets (high frequency) are used for input data among father wavelets (low frequency) and disassembled mother wavelets (high frequency). Second, reinforcement learning is performed on the input data. Here, the reinforcement learning network uses LSTM. Depending on the reinforcement learning method, the action is determined by the LSTM network or it is determined randomly. Third, we update and study the deep reinforcement learning network to demonstrate the performance of a transaction by changing the state according to the action of a given transaction and to learn the optimal investment transaction system using appropriate rewards.

Preprocessing Input Data In machine learning, preprocessing the input data can influence the learning outcome. In particular, financial time-series data may not be suitable for using raw data for learning because such data have complexity, incompleteness, and abnormality. Rather than using Fourier transform, which cannot explain information about the time resolution, we remove noise via WT of the input stock time-series data. WT increases the temporal and frequency resolutions of the high-frequency component signal. In contrast, for low-frequency component signals, the frequency resolution is increased and time resolution is

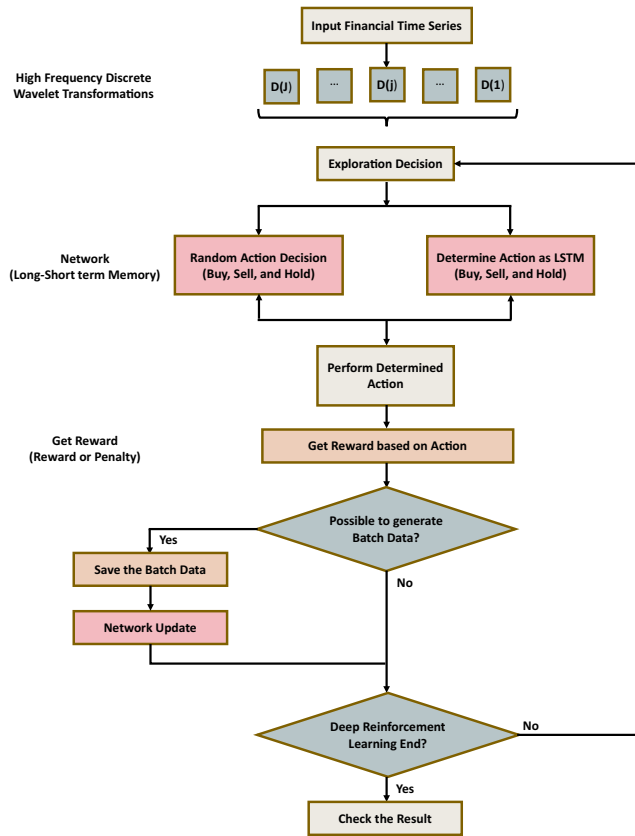


Fig. 2 Flowchart of HW_LSTM_RL structure

reduced. High frequencies (mother wavelets) describe the details of the time-series data, while low frequencies (father wavelets) represent the overall waveform that is represented smoothly. Here, we invest a certain amount of money and train the trading system to make a profit; thus, we required detailed and noiseless signal information. If we seek to predict the overall shape of the stock price going increases and decreases, we can use the WT data by removing noise from the data. However, we want to profit even when the stock price increases, and we require detailed information in the time-series data to obtain the least damage or profit even when the stock price is reduced. Therefore, in the first step, we employ only the high frequency as input data among the data from which noise is removed via WT.

Action Decision In this step, high-frequency data decomposed by WT are used as the input data for reinforcement learning, and an action is determined through exploration or is determined by the network. Here, actions include buy, sell, and hold.

If we decide the action randomly, it is determined by decaying ϵ -greedy. In other words, the more us progress through the step, the less random the probability of

choosing. For example, if $\epsilon = 0.5$, when first epoch, half of the period is not learned, and the action is determined randomly. However, when the step progresses and learning is performed, learning the action randomly with the same value of 50% is not an efficient way to learn the trading system. Therefore, as the step progresses, the value of ϵ decreases gradually; thus, the random value decreases when determining actions. The decaying ϵ -greedy is expressed as follows:

$$\epsilon \leftarrow \epsilon / (t + 1), \quad (11)$$

where, for any t -step, which reduces the probability that ϵ depends on randomness as the iteration becomes smaller.

When determining the action by the network, the reinforcement learning network uses LSTM. As we use reinforcement learning (rather than supervised learning), we do not use the softmax function value in the final output gate step. Here, we determine the behavior of the next step through the maximum reward. However, the reason for calculating the probability value using the softmax function determines the stock quantity when buying or selling stocks. At the time of investment trading, the maximum number of simultaneous trades is multiplied by the probability value determined by the softmax function. Here, the approximator of the output is the same as in (12) in Network Update of Section 3.3, which will be explained in detail in Network Update of Section 3.3.

Network Update The final step involves performing deep reinforcement learning (deep RL) to improve the performance of transactions through status change and the action process according to the reward of the given transaction. In other words, it will be rewarded immediately according to the set reward rate. In addition, if the percentage of profits or losses in the portfolio exceeds the rate at which batch data must be generated, more batch data are generated to update the network, and this process leads to learning.

The goal of our agent is to maximize future rewards. Here, we define the optimal action-value function $Q^*(s, a)$ as the maximum expected return that can be achieved by looking at some sequence, taking some action a , and following some strategy. If the optimal value $Q^*(s', a')$ of sequence s' the next time step is known for all possible actions a' , then the optimal strategy is to select action a' that maximizes the expected value of $r + \gamma Q^*(s', a')$. The optimal action-value function is defined as follows:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{D}'} [r + \gamma \max_{a'} Q^*(s', a') | s, a], \quad (12)$$

where $\mathcal{D}'$ is a probability distribution over sequences of states s and action a , r is the immediate reward from taking action a and transitioning from state s to state s'

and γ is the discount factor that weights the importance of future rewards versus immediate rewards. In the next step, if $Q^*(s, a)$ of the sequence of next states s' is known for all possible action a' , the optimal strategy is to select action a' to maximize the expected value of $r + \gamma Q^*(s', a')$.

The basic concept of many reinforcement learning algorithms is to use the Bellman equation as an iterative update to estimate the action-value function. The Q-function update can be learned using a value iteration approach as follows:

$$Q_{k+1}(s, a) = \mathbb{E} [r + \gamma \max_{a'} Q_k(s', a') \mid s, a]. \quad (13)$$

This value iteration algorithm converges to the optimal action-value function $Q_k \rightarrow Q^*$ as $k \rightarrow \infty$ in Sutton & Barto [40]. However, this basic approach is completely impractical because the action value function is estimated individually for each sequence without generalization. Instead, it is common to use a function approximator to estimate the action-value function $Q(s, a; \theta) \approx Q^*(s, a)$. The Q-network can be trained by minimizing a series of loss functions $\mathcal{L}_k(\theta_k)$ that vary at each iteration k :

$$\mathcal{L}_k(\theta_k) = \mathbb{E}_{s, a \sim \mathcal{D}'} [(y_k - Q(s, a; \theta_k))^2], \quad (14)$$

where $y_k = \mathbb{E}_{s' \sim \mathcal{D}} [r + \gamma \max_{a'} Q(s', a'; \theta_{k-1}) \mid s, a]$ is the target for iteration k , and $\mathcal{D}'$ is a probability distribution over sequences s and actions a . When optimizing the loss function $\mathcal{L}_k(\theta_k)$, the parameters of the previous iteration θ_{k-1} are fixed. The loss function is differentiated in terms of weights reaching the next gradient as follows:

$$\nabla_{\theta} \mathcal{L}_k(\theta_k) = \mathbb{E}_{s, a \sim \mathcal{D}'; s' \sim \mathcal{D}'} [(r + \gamma \max_{a'} Q(s', a'; \theta_{k-1}) - Q(s, a; \theta_k)) \nabla_{\theta_k} Q(s, a; \theta_k)]. \quad (15)$$

Optimization of the loss function is weighted in every step, and the expected value is replaced by a single sample of the action distribution $\mathcal{D}'$ as Watkins & Dayan [42]. Here, we use the Adam optimizer. This algorithm is model-free; thus, it finds the action by the estimated ϵ -greedy action selection. Learn about greed strategy $a = \max_a Q(s, a; \theta)$ and follow the distribution of behaviors to properly navigate the main space. In practice, the behavior distribution is often selected by an ϵ -greedy strategy that follows the greedy strategy of probability $1 - \epsilon$ and selects a random behavior according to probability ϵ .

The pseudocode of the proposed HW_LSTM_RL structure is described as follows. Here, we explain the framework by dividing it into three steps. Not that steps 2 and 3 are reinforcement learning; thus, we write it without distinction pseudocode in algorithm 1.

Algorithm 1 The HW_LSTM_RL algorithm.

Input: Indices data of Open, Close, Volume, MA5_Close, and MA10_Close

for $j \in 1, \dots, n$ **do**

 Perform DWT as much as wavelet levels n

return High frequencies data $D(1), D(2), \dots, D(j)$

Input: Stock Data Information (Wavelet levels 2-high frequency data of Open, Close, Volume, MA5_Close, and MA10_Close), Agent state Information (Changed stocks ratio, and the return of Portfolio value), action-value function Q networks and it parameters θ , and ϵ

 Initialize replay memory $\mathcal{D}'$ to capacity N

 Initialize action-value function Q with random weights

for episode = 1, $\dots$, M **do**

 Initialize a random process ϵ for action exploration

 Receive initial Stock information state s_1

for $t = 1, \dots, T$ **do**

 Select $a_t = \max_a Q^*(s_t, a; \theta)$

 Execute action a_t in emulator and observe reward r_t and next state s_{t+1}

 possible to generate batch data

 Generate batch data (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}'$

 Sample minibatch of transitions

(s_k, a_k, r_k, s_{k+1}) from $\mathcal{D}'$

 Set

$$y_k = \begin{cases} r_k & \text{for terminal} \\ r_k + \gamma \max_{a'} Q(s', a'; \theta_{k-1}) & \text{for non terminal.} \end{cases}$$

 Update by minimizing the loss:

$$\mathcal{L}_k(\theta_k) = \mathbb{E}_{s, a \sim \mathcal{D}'} [(y_k - Q(s, a; \theta_k))^2]$$

 loss function in terms of weights

 reaching the next gradient: $\nabla_{\theta} \mathcal{L}_k(\theta_k)$

$$= \mathbb{E}_{s, a \sim \mathcal{D}'; s' \sim \mathcal{D}'} [(r + \gamma \max_{a'} Q(s', a'; \theta_{k-1}) - Q(s, a; \theta_k)) \nabla_{\theta_k} Q(s, a; \theta_k)]$$

end for

end for

3.4 Structure of portfolios

3.4.1 Structure of trading systems

We consider agents that trade fixed position sizes in a single security. By trading different amounts of securities, we can generalize portfolio management for multiple assets to more sophisticated agents. A detailed description of the various asset portfolios is provided in Moody et al. [29].

Here, we assume that traders only take a certain amount of short, neutral, or long positions $F_t \in \{-1, 0, 1\}$ of constant magnitude in the risky security z_t . The position

F_t is established at the end of each time interval t and reevaluated at the end of period $t + 1$. The trading system rate of return R_t occurs at the end of time interval $(t - 1, t]$ and includes the profit or loss costs incurred at time t due to the difference in positions F_{t-1} and F_t .

As given by Moody et al. [29], the decision function to consider transaction costs and market impact is expressed as follows:

$$F_t = F(\theta_t; F_{t-1}, I_t) \text{ with } I_t = \{z_t, z_{t-1}, z_{t-2}, \dots, y_t, y_{t-1}, y_{t-2}, \dots\}, \quad (16)$$

where θ_t denotes the (learned) system parameters at time t , and I_t denotes the information set at time t , which includes the present and past values of the price series z_t and an arbitrary number of other external variables denoted y_t . A simple example is a long, short trader with $m + 1$ autoregressive inputs as follows [28]:

$$F_t = \text{sign}(uF_{t-1} + w_0r_t + w_1r_{t-1} + \dots + w_mr_{t-m} + v), \quad (17)$$

where r_t represents the price returns of z_t , and the system parameters θ are the weights $\{u, w_i, v\}$. Note that the above equation describes a discrete deterministic trader. However, we can generalize it using continuous $F(\cdot)$ (replacing sign with $\tanh$).

3.4.2 Profit for trading systems

The trading system can be optimized by maximizing performance functions $U(\cdot)$, e.g., assets and profits, and wealth functions or performance ratios, e.g., the Sharpe ratio. The Sharpe ratio is described as follows. Here, we consider two cases, i.e., addition profits and multiplication profits. The transaction cost rate (amount per share, per contract, or per dollar depending on the situation) is displayed.

Additive profits are appropriate for each trade to consider a fixed number of shares or security z_t . For example, trading a futures account or trading a standard US \$ FX contracts in dollar-denominated foreign currencies. We define $r_t = z_t - z_{t-1}$ as the *price returns* of a risky, and δ denote the transactions cost rate. The additive profit accumulated over period terminal time T with trading position size v is defined as the following *trading returns*, R_t , as Moody & Saffell [28]:

$$P_T = \sum_{t=1}^T R_t, \quad \text{where} \\ R_t = v\{F_{t-1}r_t - \delta|F_t - F_{t-1}|\}, \quad (18)$$

with $P_0 = 0$ and typically $F_T = F_0 = 0$. The wealth of the trader is defined as $W_T = W_0 + P_T$.

Multiplicative profits, a fixed fraction of accumulated wealth $\mu > 0$ is applied to investing in each long or short trade. For example, it is leverage in FX trading. Define $r_t = z_t/z_{t-1}$. If short sales are not allowed and the leverage factor is fixed at $\mu = 1$, then the assets at expiration time T are like Moody & Saffell [28]:

$$W_T = W_0 \Pi_{t=1}^T \{1 + R_t\}, \quad \text{where} \\ \{1 + R_t\} = \{1 + F_{t-1}r_t\}\{1 - \delta|F_t - F_{t-1}|\}. \quad (19)$$

3.4.3 Optimization of traders and portfolios via deep recurrent reinforcement learning

After determining the action randomly or by learning, we apply the reward of the action and transaction optimization to the deep reinforcement learning, as shown in Fig. 2.

The reinforcement learning algorithm proposed by Bertsekas et al. [5] provides an approximate solution to the solution of optimal control of probabilistic dynamic programming problems through online. Reinforcement learning adjusts the system parameters to maximize the expected rewards caused by the system's actions, which is achieved through trial and error exploration of the environment. Here, there is no model to learn, and we receive reinforcement signals through information about whether our actions are good or bad in the given environment (i.e., via a reward).

We consider that the performance functions of profit or wealth $U(W_t)$ after a sequence of T time steps. More generally of the whole time sequence of trades as follows:

$$U(W_1, W_2, \dots, W_t, \dots, W_T), \quad (20)$$

where $t \in [1, T]$, and T is a terminal time.

$U(W_T)$ is the general form for path-dependent performance functions like the Sharpe ratio. The performance function at time T can be expressed as a function of the sequence of the trading returns as follows:

$$U(R_1, R_2, \dots, R_T) \quad (21)$$

In future, we will simply use the (21) as U_T .

The goal is to maximize U_T for trading system model $F_t(\theta)$. To accomplish this, we must adjust parameter θ . Thus, we generalize to a vector case for a portfolio of gradient relative to θ of U_T [28]:

$$\begin{aligned} \frac{dU_T(\theta)}{d\theta} &= \sum_{t=1}^T \frac{dU_T}{dR_t} \left\{ \frac{dR_t}{dF_t} \frac{dF_t}{d\theta} + \frac{dR_t}{dF_{t-1}} \frac{dF_{t-1}}{d\theta} \right\} \\ &= \sum_{t=1}^T \frac{dU_T}{dR_t} \left\{ \frac{dR_t}{dF_t} \frac{dF_t}{d\theta_1} + \dots + \frac{dR_t}{dF_t} \frac{dF_t}{d\theta_J} \right. \\ &\quad \left. + \frac{dR_t}{dF_{t-1}} \frac{dF_{t-1}}{d\theta_1} + \dots + \frac{dR_t}{dF_{t-1}} \frac{dF_{t-1}}{d\theta_J} \right\}, \end{aligned} \quad (22)$$

where $\theta = [\theta_1, \dots, \theta_J]$, and θ is generally a vector with J parameters. The above expression written in scalar F_t can be applied by backpropagating the reinforcement signal in a manner similar to the Adaline learning rule [44] with a pre-threshold output to the traders of a single risk asset.

On the forward pass, the data can be optimized in batch mode by repeatedly calculating U_T values and adjusting the trading system parameters using gradient ascent or other optimization methods.

$$\Delta\theta = \rho \frac{dU_T(\theta)}{d\theta}, \quad (23)$$

where ρ is learning rate. other optimization methods. Due to the unique recurrence, the quantity $dF_t/d\theta$ are the *total* derivative that depends on the entire sequence of the previous period. To calculate and optimize these total derivatives correctly in an efficient way, we need a similar approach to Back-Propagation Through Time (BPTT) Rumelhart et al., and Werobos et al. [34, 43]. The temporal dependence in a series of decisions is explained by a recursive update equation on the parameter gradients defined is :

$$\frac{dF_t}{d\theta} = \frac{\partial F_t}{\partial \theta} + \frac{\partial F_t}{\partial F_{t-1}} \frac{\partial F_{t-1}}{\partial \theta_{t-1}}, \quad (24)$$

where $\theta = [\theta_1, \dots, \theta_J]$, θ is generally vector with J number of the parameters.

Equations (22) and (24) assume differentiation of F_t . In the case of a short-term deal, the reinforcement signal can be propagated back through the pre-threshold output in a manner similar to the Adaline learning rule [44]. Likewise, the same can be said of Adam optimizer [19], which added momentum to Adaline. Equations (22), (23) and (24) constitute a batch deep reinforcement learning algorithm.

Considering only the term of (22), which depends on the most recently realized rate of return R_t during data passing, we can get a stochastic optimization for online as follows as Moody & Saffell [28]:

$$\frac{dU_t(\theta)}{d\theta_t} \approx \frac{dU_t}{dR_t} \left\{ \frac{dR_t}{dF_t} \frac{dF_t}{d\theta_t} + \frac{dR_t}{dF_{t-1}} \frac{dF_{t-1}}{d\theta_{t-1}} \right\}, \quad (25)$$

where $\theta = [\theta_1, \dots, \theta_J]$, θ_t is generally vector with J number of the parameters.

Then update online with parameters :

$$\Delta\theta = \rho \frac{dU_t(\theta_t)}{d\theta_t}. \quad (26)$$

This algorithm performs stochastic optimization because the system parameter θ_t changes through the training data

during each forward pass. The stochastic on-line analog of (24) is follows:

$$\frac{dF_t}{d\theta_t} \approx \frac{\partial F_t}{\partial \theta_t} + \frac{\partial F_t}{\partial F_{t-1}} \frac{\partial F_{t-1}}{\partial \theta_t}, \quad (27)$$

where $\theta_t = [\theta_{1t}, \dots, \theta_{Jt}]$, θ_t is generally vector with J number of the parameters.

3.5 The sharpe ratio

Nowadays, investors are trying to maximize risk-adjusted returns as suggested by modern portfolio theory. The Sharpe ratio is the most widely used measure of risk-adjusted return as Sharpe [35]. The Sharpe ratio is defined as follows :

$$S_T = \frac{\text{Average}(R_t)}{\text{Standard Deviation}(R_t)}, \quad (28)$$

where R_t is the trading system returns for period t , the average and standard deviation are estimated over returns for periods $t = \{1, \dots, T\}$. We can train our trading system to maximize the Sharpe ratio as follows. First, we define the Sharpe ratio as Moody et al. [29] for n returns R_i as an estimate of the first and second moments in the returns' distribution :

$$S_n = \frac{A_n}{K_n(B_n - A_n^2)^{1/2}} \quad (29)$$

where $A_n = \frac{1}{n} \sum_{i=1}^n R_i$, $B_n = \frac{1}{n} \sum_{i=1}^n R_i^2$, and $K_n = \left(\frac{n}{n-1}\right)^{1/2}$. The normalizing factor K_n is required for estimating the bias of the standard deviation, but it is not relevant to the optimization. The derivative associated with the system parameter is:

$$\begin{aligned} \frac{dS_n(\theta)}{d\theta} &= \sum_{i=1}^n \left\{ \frac{dS}{dA_n} \frac{dA_n}{dR_i} + \frac{dS}{dB_n} \frac{dB_n}{dR_i} \right\} \\ &+ \left\{ \frac{dR_i}{dF_i} \frac{dF_i}{d\theta_1} + \dots + \frac{dR_i}{dF_i} \frac{dF_i}{d\theta_J} \right. \\ &\quad \left. + \frac{dR_i}{dF_{i-1}} \frac{dF_{i-1}}{d\theta_1} + \dots + \frac{dR_i}{dF_{i-1}} \frac{dF_{i-1}}{d\theta_J} \right\} \\ &= \sum_{i=1}^n \left\{ \frac{B_n - A_n R_i}{K_n(B_n - A_n^2)^{3/2}} \right\} \\ &+ \left\{ \frac{dR_i}{dF_i} \frac{dF_i}{d\theta_1} + \dots + \frac{dR_i}{dF_i} \frac{dF_i}{d\theta_J} \right. \\ &\quad \left. + \frac{dR_i}{dF_{i-1}} \frac{dF_{i-1}}{d\theta_1} + \dots + \frac{dR_i}{dF_{i-1}} \frac{dF_{i-1}}{d\theta_J} \right\}, \quad (30) \end{aligned}$$

where $\theta = [\theta_1, \dots, \theta_J]$, θ is generally vector with J number of the parameters. The above expression, written

in scalar F_t , can be applied to traders of a single risk asset described in Section 3.4.1 by backpropagating the reinforcement signal through a pre-threshold output.

In the forward passes, we can optimize the system in batch mode by repeatedly calculating the U_T value, passing the data and adjusting the trading system parameters using gradient descent or other optimization methods as follows:

$$\Delta\theta = \rho \frac{dU_T(\theta)}{d\theta}. \quad (31)$$

4 Design of data and experiment

4.1 Data descriptions

This section describes the input variables and data resources selected to train the sample selection and trading system. Here, we selected the following stock indices: S&P500 index, Dow Jones Industrial Average Index (DJI), and KOSPI200 index. As mentioned previously, market conditions can affect reinforcement learning. A stock index measures a subset of the stock market and helps investors calculate market performance by comparing the current price level to previous prices. Learning a trading system using an index has a problem, i.e., it appears that only a single stock is traded; however, since stocks included in the index can be changed rather than maintained in the same period, the transparency and stability of the experiment will be improved. We used an index for that. To compensate the single-stock trading problem, we tested the indexes on the S&P500, DJI, and KOSPI200 indices over various periods.

The S&P500 and DJI indices are traded on the New York Stock Exchange, which is considered the most advanced global financial market. Therefore, it represents the market with the highest level of development. The KOSPI200 index represents the market situation between developed and developing markets. The Republic of Korea's financial markets are high in foreign investors, highly volatile, and heavily influenced by other countries, including the United States. Therefore, these three indices provide a natural environment for testing the proposed structure depending on market conditions. Therefore, using these three indexes, we seek to learn about investment transactions in different environments.

The proposed structure uses deep reinforcement learning with only basic input data: Open, Close, Volume, MA5, and MA10 (5- and 10-day moving average). Open, Close, and Volume whereas OHLCV (Open, High, Low, Close, and Volume) was not used for input data because it is believed to have a value that disturbs investment in trading system learning. For MA20, MA60, and MA120, a trend may emerge in the graph. Therefore, we decomposed the

given data through WT and decided that MA20, 60, and 120 were unsuitable for the proposed structure, which only uses high-frequency with detailed information. Therefore, we only used up to MA5 and MA10. The values for Open, Close, and Volume for the three indices were downloaded from Yahoo Finance (<https://finance.yahoo.com>). However, MA5 and 10 were calculated from the Close value.

4.2 Experiment design

The proposed structure and compared structures are defined as follows.

The experiment was designed using Keras and implemented in Python 3.5. We also experimented with a GPU graphics card equipped with TITAN 5. Note that we referred to Github when designing the structure of the reinforcement learning [45].

We divided the experiment into seven. Depending on how the action is determined, it was divided into {LSTM_RLs}, {RNN_RLs}, and Buy and Hold. Here, Buy and Hold represent the first purchase of the maximum number of shares equivalent to maximum the amount held in our account, followed by a hold during the test period. The Buy and Hold method offers several investment benefits. Investors using the Buy and Hold strategy actively choose to invest, but don't have to worry about short-term price movements and technical indicators. It is also an ideal buying and holding method for individuals seeking healthy long-term benefits. We did not compare the experimental results with the portfolio selection system using the calculated expected return of the portfolio or the risk variance of the portfolio because the experimental data contained 500 stocks in the S&P500 index itself. However, in our experiment, only the Index was used. Therefore, the comparison value for the investment transaction was set as the Sharpe ratio of the seven experimental results including Buy and Hold.

{LSTM_RLs}, {RNN_RLs} were also divided into three types. First, only high frequency was used among the data decomposed by WT. Second, both high and low frequencies were used as data decomposed by WT. Third, WT was not used, original data. Therefore, we conducted a total of seven experiments, including Buy and Hold.

In the next section, we describe the experimental design of data settings, agents, environments, and learning.

Training & Test sets The procedure for learning a trading system was described by Bao et al. [2], and configured by M'ng & Mehralizadeh [30]. However, we did not use the validation part in this experiment; thus, it comprises two parts. The first part is training the model and updating its parameters. The second part is testing to predict the data using the best structure. Here, due to the limited data, we

Table 1 A total of twelve sets of periods for three-years of training and one-year of testing

Year	Training set	Test set
Year 1	July 01, 2004 ~ December 31, 2006	January 1, 2007 ~ December 31, 2007
Year 2	July 01, 2005 ~ December 31, 2007	January 1, 2008 ~ December 31, 2008
Year 3	July 01, 2006 ~ December 31, 2008	January 1, 2009 ~ December 31, 2009
Year 4	July 01, 2007 ~ December 31, 2009	January 1, 2010 ~ December 31, 2010
Year 5	July 01, 2008 ~ December 31, 2010	January 1, 2011 ~ December 31, 2011
Year 6	July 01, 2009 ~ December 31, 2011	January 1, 2012 ~ December 31, 2012
Year 7	July 01, 2010 ~ December 31, 2012	January 1, 2013 ~ December 31, 2013
Year 8	July 01, 2011 ~ December 31, 2013	January 1, 2014 ~ December 31, 2014
Year 9	July 01, 2012 ~ December 31, 2014	January 1, 2015 ~ December 31, 2015
Year 10	July 01, 2013 ~ December 31, 2015	January 1, 2016 ~ December 31, 2016
Year 11	July 01, 2014 ~ December 31, 2016	January 1, 2017 ~ December 31, 2017
Year 12	July 01, 2015 ~ December 31, 2017	January 1, 2018 ~ December 31, 2018

adopted the time frame described by Bao et al [2], and M'ng & Mehralizadeh [30] in each part.

The experiment comprised sets of one- and three-year testing periods. For the one-year testing period, the S&P500, DJI, and KOSPI200 indices were tested for the same period. For the three-years period, only the S&P500 index was tested.

First, for the one-year period, each set comprised 30 months of training and 12 months of testing, with a total of 12 sets. For example, the first set (Year 1 in Table 1) studied the training set for 30 months (July 1, 2004 to December 31, 2006), and the test was conducted over the entire 2007 period. Similarly, the second set (Year 2 in Table 1) tested 2008. As shown in Fig. 3, the test period was shifted by 1 year to constitute a total of 12 sets. For the three-years test period, each set comprised seven-years of training (84 months) and three-years of testing (36 months), with a total of two sets. For the three-years test period, as shown in Table 2, shifted by 3 years was composed of a total of 2 sets.

Figure 3 shows the composition of the training and test periods, and Table 1 shows the 12 sets of training and testing periods for the S&P500, DJI, and KOSPI200 indices.

Table 2 shows the results of testing for only the S&P500 index with two sets of training periods and testing periods.

The numbers after the training set or test set can be considered the numbers after the Year. For example, training set 2 represents July 01, 2005 to December 31, 2007, which is the training set period for Year 2. In addition, test set 2 corresponds to January 1, 2008 December 31, 2008 of Year 2.

- Year number = Training set number + Test set number

4.3 Reinforcement learning settings

4.3.1 Data input in the environment

The input data vary according to the structure of reinforcement learning for each transaction. In Fig. 1, the signal of $f(t)$ is expressed as low frequency $A(j)$ in red and high frequency $D(j)$ ($j = 1, \dots, J$) in blue, where J is the level of the resolution, as shown in (7).

Note that the input data were the same for both LSTM and RNN. It comprised Open, Close, Volume, MA5 (five-day average closing price), and MA10 (ten-day average

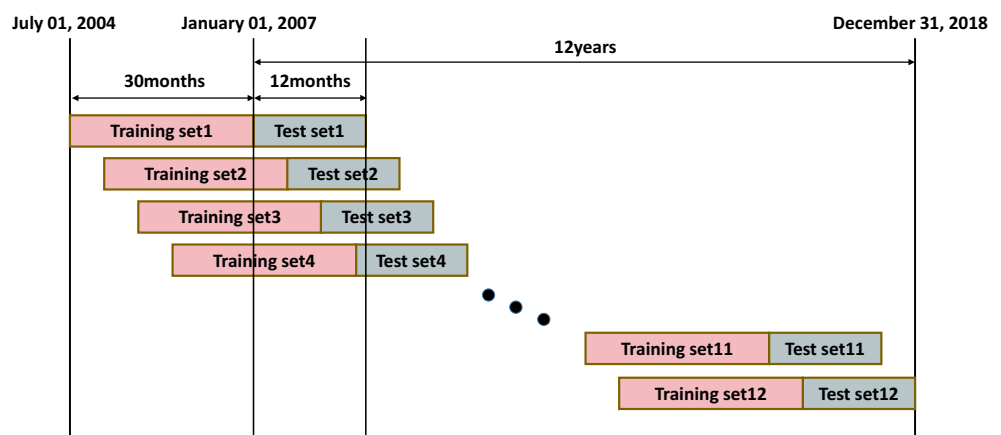
Fig. 3 Continuous dataset arrangement for training and testing during the entire sample period

Table 2 A total of two sets of periods for seven-years of training and three-years of testing

Year	Training set	Test set
Year 13	January 1, 2006 ~ December 31, 2012	January 1, 2013 ~ December 31, 2015
Year 14	January 1, 2009 ~ December 31, 2015	January 1, 2016 ~ December 31, 2018

closing price). However, according to each structure, HW_LSTM_RL and HW_RNN_RL used only high frequency, and W_LSTM_RL and W_RNN_RL used both high and low frequencies. Finally, LSTM_RL and RNN_RL used the data without WT.

To learn the trading system via reinforcement learning, in addition to the input data, we require information about the agent. To provide information about the agent, the number of changed stocks ratio and the return of the portfolio value were given as input values. The input data are shown in Table 3. This is the environment for reinforcement learning. In this environment, the agent manages chart data for stocks to invest. The agent has a chart of data in the environment because the goal is to return and invest, and then learn from the results; however, it provides data sequentially from the past to the most recent. In other words, an agent that has gone back in time will not know the future chart day.

When the high-frequency input data were used as input, they choose whether to decide the investment behavior randomly or through the network. If the decision is made randomly, we select to buy, sell, or hold. When the decision is made through learning using an artificial neural network, the action of reinforcement learning investment is decided. We proposed to buy, sell, and hold an artificial neural network through LSTM. The number of cells in {LSTM_RLs} and {RNN_RLs} were 256, 512, and 256 in each LSTM and RLL layer. In {LSTM_RLs} and {RNN_RLs}, the activation function used ReLU, the Adam optimizer was used, and the learning rate was 0.001. In addition, the keep-probability was 0.8, and the number of epochs was 500. The only difference between the proposed structure and the compared structures was the input data, as shown in Table 3. Here, HW_LSTM_RL, and HW_RNN_RL

had 16 input dimensions, W_LSTM_RL, and W_RNN_RL had 23 input dimensions, LSTM_RL, and RNN_RL had 9 input dimensions, and the output layer had 3 dimensions. The input layer combined the 14-dimensional learning data and 2-dimensional data representing the agent status, and the output layer comprised three dimensions, i.e., the buy, sell, and hold actions.

4.3.2 Agent

An agent is an investor that buys, sells or holds shares. The agent considers the following: initial portfolio value, cash balance remaining from holding stocks in the initial portfolio value, and the number of stocks. The valuation of the cash balance and stock current stocks price adds up to the portfolio value

$$PV = \text{Number of Stocks} \times \text{Current Stocks Price} + \text{Cash Balance}$$

If the portfolio value is higher than the initial portfolio value, profit has been obtained. If the portfolio value is low, loss has occurred. The investor's goal is to increase the portfolio value. When an agent buys, the number of stocks, held increases, and cash balance are reduced. In contrast, when an agent sells, the number of held stocks is reduced and the cash balance increases.

The agent also includes transaction taxes incurred while trading and fees incurred when buying and selling. The transaction cost to securities companies was taken 0.05% as Moody et al. [28]. Note that this part can be modified to suit the given situation.

Table 3 Input data according to the structures of the trading system

	High frequency wavelet transform HW_LSTM_RL HW_RNN_RL (Input data : 16)	Wavelet transform W_LSTM_RL W_RNN_RL (Input data : 23)	No wavelet transform LSTM_RL RNN_RL (Input data : 9)
Stock data Information	Open, Close, Volume, MA5_Close, MA10_Close, MA5_Volume, MA10_Volume		
	Wavelet Level 2 : Using D(1), D(2), then $2 \times 7 = 14$ inputs	Wavelet Level 2 : Using A(2), D(1), D(2), then $3 \times 7 = 21$ inputs	No Wavelet : 7 inputs
Agent State Information	2 inputs : Changed Stocks ratio, the return of Portfolio value		

The actions an agent can take include buy, sell, and hold. Agents treat these actions as constants by assigning them specific values. The action that gives maximum value is taken as the optimal action value function. We did not consider buying credit by borrowing money even if the cash balance was insufficient or by selling it in advance even if we do not own the stock, and buying it later. Therefore, if we decided to buy but the cash balance was insufficient, we held. In contrast, if a sale was decided but the number of stocks could not be held, the hold action was performed. Note that hold includes the value given by the determined hold action and the above case.

In addition, calculating the probability value using the softmax function determines the stock quantity when buying or selling stocks; thus, stocks were traded as many times as the probability value multiplied by the maximum number of stocks that can be traded at one time. If we hold it, the number of held stocks or the cash balance is unaffected; however, the portfolio value may change due to potential price fluctuations. Buying and selling change both the portfolio value and the number of held shares.

For the S&P500, DJI, and KOSPI200 the initial portfolio was valued at 10 million. When buying or selling stocks, the minimum transaction unit that could be traded simultaneously was 1, and the maximum transaction unit was 1000.

If an action was determined by exploration rather than policy network, the exploration rate was 50%. Here, as the number of epochs increased, the decaying ϵ -greedy method in (11) was used as the exploration method.

4.3.3 Reward

Note that we did not build a model. Instead, we modified the state and action determined by actions and the reward. Therefore, appropriate rewards should be given based on the actions taken and under what conditions the actions were taken. Two types of rewards were considered, i.e., immediate and delayed rewards. These rewards are based on profit and loss. For example, according to the reward ratio, if the profit and loss rate was 10% and the profit rate reached 10%, the previous investment action was considered to be positive. If the loss rate was 10%, the investment action was considered to be negative. We set the immediate and delayed rewards differently according to the given situation. In addition, we compared the set profit and loss ratios with more than or less than a given ratio. The following rewards were used in the experimental groups:

[Experimental group] : Immediate Rewards

- If portfolio value $\leq$ initial portfolio value,
if profit and loss rate ≤ 0 ,
then immediate reward is -5.

if profit and loss rate > 0 ,
then immediate reward is 1.

- If portfolio value $>$ initial portfolio value,
if profit and loss rate $> 0.5\%$,
then immediate reward is 3.
- if $0 < \text{profit and loss rate} \leq 0.5\%$,
then immediate reward is 2.
- if profit and loss rate ≤ 0 ,
then immediate reward is -3.

[Control group] : Immediate Rewards

- If profit and loss rate > 0 , then 1.
- Otherwise is -1.

[Experimental group] : Delayed Rewards

- If profit and loss rate $> 3 \times$ generated batch data (GBD) rate,
then delayed reward is 3.
- If $1 \times \text{GBD rate} < \text{profit and loss rate} \leq 3 \times \text{GBD rate}$,
then delayed reward is 1.
- If $-3 \times \text{GBD rate} < \text{profit and loss rate} \leq -1 \times \text{GBD rate}$,
then delayed reward is -2.
- If profit and loss rate $\leq -3 \times \text{GBD rate}$,
then delayed reward is -3.
- else otherwise,
then delayed reward is 0.

[Control group] : Delayed rewards

- If profit and loss rate > 0 , then 1.
- Otherwise is -1.

Immediate rewards lead to judging first or below the initial portfolio value because, if the stock market is bearish, there will be cases where the stock is held at the portfolio value without trading. To prevent this, the first criterion was used. If we do not invest in stocks in the bear market, we do not lose the cash we possess; however, the goal is to learn stock trading.

The profit and loss rates are determined after determining the portfolio value according to the initial portfolio value. When the portfolio value is less than the initial portfolio and if the profit and loss rate is also small, the immediate reward is 5. This is done to make the learning very wrong. When the portfolio value is greater than the initial portfolio, the profit loss rate is greater than 0 and greater than 0.5%, which is divided separately to compensate for the learning effect.

Investing more than a percentage of the profits or losses in stocks creates situational and actional learning data. These training data are applied all at once to update the policy LSTM (or RNN) network. This method of learning is referred to as batch learning. In our experiment, we

set the value of the generated batch data (GBD) rate to 1%. Although it generates batch data, the GBD rate also determines the delayed reward value. When a delayed reward occurs, the current delayed reward is applied to all actions performed between the previous delayed reward and the current delayed reward. Here, the reason for applying the present discount factor is blurred as it goes back to the past. The discount factor was set 0.99.

The GBD rate includes training data samples, agent actions, memory containing immediate rewards, the size of the batch data to be generated, the discount factor, and the delayed reward. Then, the learning progresses with the prepared batch data such that learning can be updated with the GBD rate.

In addition, immediate reward and rewards are provided at 1% and 3% to compensate for more profit and apply more penalties for more losses. This is why rewards and penalties are set to 5 and 3 rather than 1 and -1. In other words, there are higher penalties for losses. Its metric represents people's investment sentiments. When trading stocks, if we observe a loss of approximately 25%, we think that the profit on that loss must be approximately 33%; thus, the investment is going to the same value. That is, it is more sensitive to loss than profit. For losses, this is done to learn from incorrect actions as the penalty value increases. Thus, different profit and penalty rewards are provided. In this experiment, we compared the control group, where the reward is the same ratio (the current ratio), to the other ratios (1 and -1).

We conducted tests on the experimental and control groups on only the S&P500 index. For the DJI and KOSPI200 indices, we conducted an experiment with the proposed rewards for the experimental group. We found that the experimental group suggested better results than the control group under the same conditions. Therefore, it is very important to set the reward carefully according to the given environment and conditions because it can affect the result of reinforcement learning.

5 Experimental results

After decomposing into high and low frequencies via WT, the input data were composed (Table 3) in the proposed and compared structures, and compared to the S&P500, DJI, and KOSPI200 indices according to the Sharpe ratio. Tables 4 through 6 show the Sharpe ratios for the S&P500, DJI, and KOSPI200 indices for each 30-months training period and 12-months test period from Years 1 to 12, and Table 7 shows the Sharpe ratio for a seven-years training period and three-years testing period for Years 13 and 14, respectively, as in Table 2 for the S&P500 index. Tables 8 and 9 show the daily and cumulative returns for the S&P500 index data for the one-year test period, and Table 10 shows the

difference in Sharpe ratio when the reward was divided into the experimental group and control groups (Section 4.3.3). Figure 4 shows the portfolio values for Years 2 and 12.

The Sharpe ratio is an indicator that evaluates a fund's performance using standard deviations and is an indicator of the extent of excess returns earned by a particular fund from investing in a unit of a risky asset. If the Sharpe ratio is greater than 0, it is "possible to invest." If the Sharpe ratio is great than 1, the investment is considered to be "good".

Through these experiments, the proposed structure demonstrated following characteristics.

- Markets with higher volatility eliminate risk and have a higher Sharpe ratio than other similar structures or Buy and Hold. The proposed structure is well-suited to volatile markets, e.g., the diversified financial market. The use of high frequencies decomposed by WT into input data accounts for the detailed volatility. The second highest value after HW_LSTM_RL in the tables is HW_RNN_RL; thus, high-frequency information has an important effect on investment transactions.
- Although we used limited training data, the proposed HW_LSTM_RL structure is useful for investment trading. In addition, the training data show a modest upward trend; thus, even if the learning data are trained without information about variability, the high-frequency information of the input data given in the volatile field can prepare for the volatility.
- The proposed HW_LSTM_RL structure also demonstrates the importance of the composition of the input data, the learning network, and the reward composition. To improve the learning performance in deep reinforcement learning, proper compensation for data preprocessing, a data processing network, and each learning scenario should be designed carefully.

In Tables 4 to 10, the highest values in each row are shown in bold (by Year).

According to the Sharpe ratio values for the S&P500 index shown in Table 4, the proposed HW_LSTM_RL has a higher Sharpe ratio than other similar structures and Buy and Hold, yielding an average of approximately 0.8827. Although it does not exceed 1, it has a higher value than other similar structures. The Sharpe ratio values for DJI in Table 5 demonstrate that the proposed HW_LSTM_RL structure has the highest value (greater than 1) with an average of 1.0548. The second highest, i.e., HW_RNN_RL, shows an average of 0.8581. The S&P500 and DJI indices already stabilized the market, and the Sharpe ratio indicates that is worth investing in the market. Table 6 shows the Sharpe ratio for the KOSPI200 index, which has an average value of 0.5968. Although this is not necessarily high, it is higher than other similar structures and higher than Buy and Hold. The KOSPI200 index is affected both by international

Table 4 Sharpe ratio for one-year test sets on S&P500 index from Years 1 to 12

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and Hold
Year 1	0.4962	0.0249	0.2776	0.3258	0.2776	0.2765	0.3059
Year 2	-0.4228	-0.8459	-0.8515	-0.8462	-0.8512	-0.8590	-0.9450
Year 3	1.3129	1.1617	1.1699	1.1920	1.1618	1.1617	0.7991
Year 4	0.8987	0.6322	0.7401	0.8362	0.7391	0.7440	0.6715
Year 5	0.1829	0.0301	0.0304	0.0302	-0.0136	0.0309	0.0677
Year 6	0.8345	0.8155	0.8014	0.8046	0.8162	0.7782	0.9395
Year 7	2.2331	2.1838	2.1562	2.1729	2.1630	2.1633	2.2287
Year 8	1.1665	1.0753	1.0754	1.0824	1.0730	1.0731	1.0894
Year 9	0.2283	0.1304	0.1306	0.1478	0.1332	0.1332	0.0324
Year 10	1.4318	1.2216	1.2204	1.0080	1.2729	1.2727	0.8860
Year 11	2.5242	2.5221	2.5202	2.5097	2.5221	2.5221	2.5897
Year 12	-0.2941	-0.4439	-0.4317	-0.3245	-0.4323	-0.4434	-0.3433
Average	0.8827	0.7090	0.7366	0.7449	0.7385	0.7378	0.6935

trends and domestic issues. In terms of the Sharpe ratio value, it is an unsatisfactory investment because it has a value of less than 1; however, given the volatility, a value of 0.5 or more is considered a successful investment.

Buy and Hold is higher than HW_LSTM_RL in Year 6 and Year 11 in Table 4, Year 10 and Year 11 in Table 5, and Year 8 and Year 11 in Table 6. However, the difference does not exceed 0.15. In this case, the Sharpe ratios are generally greater than 1 but are sometimes greater than 3. In particular, in Year 11 (test period: January 01, 2017 to December 31, 2017), the S&P500, DJI, and KOSPI200 indices all demonstrate high values for Buy and Hold because the market is a bull market with little volatility and a modest rise. The beginning of the test period was the least

expensive; thus, the Sharpe ratio for Buy and Hold, where high volumes of stocks are bought and held simultaneously, is the best portfolio value. Different results could have been obtained if the maximum number of stocks that can be held at one time was variable (we fixed the simultaneous tradeable amount to 1000 in this experiment). For Year 8 in the KOSPI200 index, the Sharpe ratio of W_LSTM_RL showed the largest value of 0; however, examining the portfolio value at the time indicates that it did not make any investments during the given test period. We closed the deal without changing the number of shares and the portfolio value. In fact, we seek to learn investment trading; thus, we cannot assess the value of doing nothing as a good investment when learning to trade. Therefore, although the

Table 5 Sharpe ratio for one-year test sets on DJI from Years 1 to 12

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and Hold
Year 1	0.6330	0.4962	0.4962	0.4046	0.4961	0.4961	0.4979
Year 2	-0.3377	-0.8590	-0.8590	-0.6396	-0.8591	-0.8591	-0.8580
Year 3	1.1307	0.7348	0.7349	1.0851	0.7346	0.7347	0.7173
Year 4	0.8602	0.6421	0.6421	0.7586	0.6421	0.6421	0.6390
Year 5	0.5468	0.3197	0.3197	0.4854	0.3196	0.3196	0.3236
Year 6	0.7154	0.5281	0.5282	0.7016	0.5280	0.5280	0.5346
Year 7	2.7287	2.1981	2.1981	2.4660	2.1983	2.1983	2.1927
Year 8	0.8606	0.7958	0.7959	0.8006	0.7956	0.7957	0.7995
Year 9	0.7300	-0.0555	-0.0561	0.3443	-0.0539	-0.0547	-0.0730
Year 10	1.1186	1.1995	1.1995	0.9470	1.1958	1.1994	1.1999
Year 11	3.2044	3.3421	3.3423	2.8786	3.3411	3.3415	3.3440
Year 12	0.4674	-0.2582	-0.2594	0.0646	-0.2594	-0.2594	-0.2594
Average	1.0548	0.7568	0.7569	0.8581	0.7566	0.7569	0.7549

Table 6 Sharpe ratio for one-year test sets on KOSPI200 index from Years 1 to 12

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and Hold
Year 1	1.8386	1.4704	1.5184	1.8092	1.5186	1.5183	1.3439
Year 2	-0.7127	-0.8718	-1.0227	-0.9959	-1.0346	-1.0428	-1.1030
Year 3	1.7699	1.7684	1.7688	1.7681	1.7688	1.7688	1.6546
Year 4	1.3609	0.0000	1.3603	1.3607	1.3605	1.3607	1.3529
Year 5	-0.2172	-0.3845	-0.3843	-0.3845	-0.3844	-0.3841	-0.3523
Year 6	1.1462	0.5853	0.5666	0.8642	0.5653	0.5656	0.6644
Year 7	0.2075	-1.0840	0.0915	0.1452	0.0915	0.0915	-0.0209
Year 8	-0.1263	0.0000	-0.1497	-0.1318	-0.1495	-0.1496	-0.2241
Year 9	0.2835	0.2809	0.2809	0.2826	0.2807	0.2815	0.2057
Year 10	0.6077	0.5861	0.5863	0.5887	0.5859	0.5857	0.5205
Year 11	2.0702	2.0451	2.0451	2.0127	2.0451	2.0449	2.1001
Year 12	-1.0666	-1.3582	-1.3582	-1.3497	-1.3588	-1.3592	-1.3097
Average	0.5968	0.2531	0.4419	0.4975	0.4408	0.4401	0.4027

value of the Sharpe ratio has the highest value of 0, this case should be excluded. Thus, except for the case where no transaction was made, the Sharpe ratio of HW_LSTM_RL is considered to be sufficiently high.

Figure 4 shows the portfolio value of the S&P500, DJI, KOSPI200 indices for Years 2 and 12. Here, x-axis represents the test period, and the y-axis represents the

portfolio value for the experiments. In Fig. 4, the black line represents the baseline with an initial investment cost of \$10 million. Among Years 1 to 12, the portfolio value is shown as a picture only when an overall poor Sharpe ratio is found.

The left figures in Fig. 4 show the learning period from July 01, 2005 to December 31, 2007 and the January 1, 2008 to December 31, 2008 test period. The 2008 test period

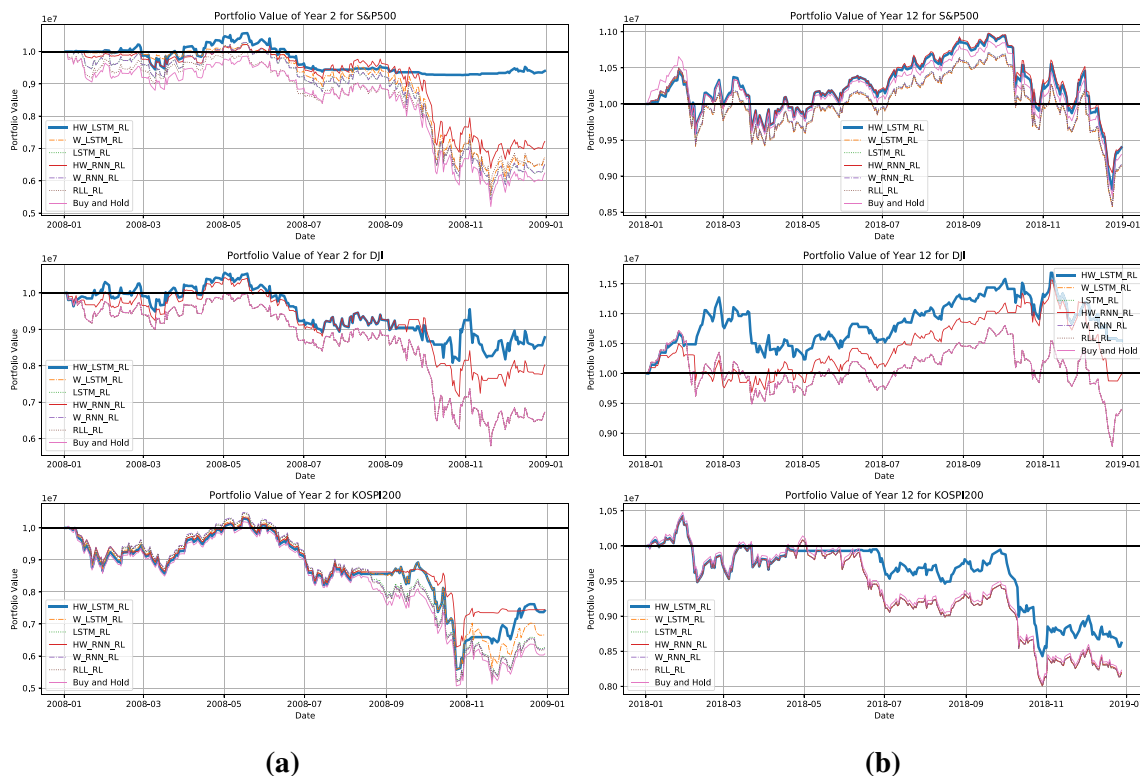


Fig. 4 Graphs in the first row are for the S&P500 index, graphs in the second row figures are for the DJI, and graphs in the third row are KOSPI200 index. **a** The left figures represent the portfolio value for Year 2. **b** The right figures represent the portfolio value for Year 12

Table 7 Sharpe ratio for three-years test sets on S&P500 index from Years 13 to 14

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and hold
Year 13	1.7373	1.6668	1.6471	1.6492	1.6468	1.6470	1.6751
Year 14	1.4372	1.2740	1.2743	1.4049	1.2806	1.2802	1.0923
Average	1.5872	1.4704	1.4607	1.5270	1.4637	1.4636	1.3837

represents the period where the US subprime mortgage suffered a global financial crisis. At this time, the leverage expansion strategy using low interest rates in the 2000s was sloppy, and the greed of the financial market for maximum profits collapsed. In the case of the S&P500 index, when there was a sharp drop between September and November, only the proposed HW_LSTM_RL structure did not cause a sharp drop in the portfolio value. Although the value of the portfolio was less than 10 million, it was greater than 9 million, and Buy and Hold, and other similar structures generally have values between 5 and 7 million. In addition, for the DJI and KOSPI200 indices, the proposed HW_LSTM_RL structure is smaller than the other structures, and the value of the portfolio falls sharply and then rises. The crash during the learning period of 2005 to 2007 was not severe; however, the proposed HW_LSTM_RL structure exhibits good results in the event of a major crash, e.g., the 2008 Lehman Brothers crisis.

The right figures in Fig. 4 show the learning period from July 01, 2015 to December 31, 2017 and the January 01, 2018 to December 31, 2018 test period. This represents the trade war period between the United States and China, which is still ongoing. The United States imposed a 25% retaliation tariff on 818 US 34 billion of Chinese imports, and

25% retaliation for US agricultural products, automobiles, and fisheries imported into China as a US 34 billion US. The trade war between the two countries began with imposing tariffs on both sides. The US-China trade war affected the US and Chinese markets, as well as the global economy, thereby making the market more volatile than ever. Even in this period, it can be seen that the proposed HW_LSTM_RL portfolio provided higher value than the compared structures or Buy and Hold because it declines less sharply and rises more sharply than the other structures. Figure 4 also shows that the proposed HW_LSTM_RL structure yields the best results in volatile or explosive markets. In addition, HW_LSTM_RL and HW_RNN_RL both used high frequency, and the graph shows that the portfolio value of HW_LSTM_RL is higher than that of the compared structures. In other words, the proposed HW_LSTM_RL structure confirms that high-frequency input data provide better results in fields with high volatility. From the results, we confirm that the proposed HW_LSTM_RL structure, which eliminates risks and improves investment profits, is suitable for investment trading in highly volatile stocks.

Table 7 shows the long-term results for the S&P500 index. For the long-term experiments, only the S&P500 index was considered. The learning and test periods are

Table 8 Daily return for one-year test sets on S&P500 index

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and hold
Year 1	0.0122	0.0079	0.0079	0.0089	0.0080	0.0080	0.0020
Year 2	-0.0214	-0.1282	-0.1375	-0.1085	-0.1374	-0.1289	-0.1536
Year 3	0.1249	0.1166	0.1170	0.1159	0.1166	0.1166	0.0861
Year 4	0.0606	0.0451	0.0520	0.0560	0.0519	0.0522	0.0480
Year 5	0.0090	-0.0016	-0.0015	-0.0016	-0.0037	-0.0013	-0.0016
Year 6	0.0419	0.0410	0.0402	0.0384	0.0410	0.0404	0.0476
Year 7	0.0940	0.0933	0.0920	0.0925	0.0923	0.0924	0.0957
Year 8	0.0488	0.0481	0.0481	0.0482	0.0479	0.0479	0.0491
Year 9	0.0122	0.0079	0.0079	0.0089	0.0080	0.0080	0.0020
Year 10	0.0671	0.0603	0.0602	0.0412	0.0623	0.0623	0.0458
Year 11	0.0662	0.0663	0.0662	0.0656	0.0663	0.0663	0.0685
Year 12	-0.0194	-0.0300	-0.0292	-0.0199	-0.0293	-0.0300	-0.0233
Average	0.0414	0.0272	0.0269	0.02879	0.0269	0.0278	0.02220

Table 9 Cumulative return on one-year test sets on S&P500 index

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_RNN_RL	W_RNN_RL	RNN_RL	Buy and hold
Year 1	3.0649	1.9766	1.9794	2.2215	2.0147	2.0140	0.5015
Year 2	-5.3855	-32.3171	-34.6474	-27.3406	-34.6285	-32.4937	-38.7019
Year 3	31.3416	29.2719	29.3767	29.0970	29.2639	29.2633	21.6220
Year 4	15.2019	11.3300	13.0400	14.0437	13.0247	13.0959	12.0507
Year 5	2.1096	0.6974	0.7027	0.6988	-0.3019	0.7151	1.5680
Year 6	10.4363	10.2054	10.0138	9.5691	10.2150	10.0585	11.8401
Year 7	23.5895	23.4145	23.1021	23.2117	23.1785	23.1851	24.0097
Year 8	12.2532	12.0640	12.0624	12.1009	12.0141	12.0150	12.3190
Year 9	3.0649	1.9766	1.9794	2.2215	2.0147	2.0140	0.5015
Year 10	16.8509	15.1286	15.1157	10.3314	15.6378	15.6366	11.4931
Year 11	16.5564	16.5634	16.5620	16.4067	16.5637	16.5639	17.1258
Year 12	-4.8492	-7.5102	-7.3055	-4.9690	-7.3173	-7.5007	-5.8213
Average	10.3529	6.9001	6.8318	7.2994	6.8066	7.0472	5.7090

shown in Table 2. When the learning period was seven years and the test period was three-years, and both the learning and test periods were longer, the proposed structure, similar structures, and Buy and Hold all obtained a value of 1 or more. In particular, the proposed structure obtained an average value of 1.5872 for HW_LSTM_RL, which is higher than other value. This result demonstrates that reinforcement learning can achieve better results when there more data are available. In addition, Tables 4–6 show that the second highest Sharpe ratio value was obtained with HW_RNN_RL when using a high frequency and not a network using LSTM. Here, the network settings were the same for both the LSTM and RNN. LSTM is known to achieve better results than RNNs. Nevertheless, the fact that HW_RNN_RL

obtained the second highest value rather than the suggested Table 2 indicates that high frequency is more appropriate for investment transactions. This also demonstrates the importance of the input data. W_LSTM_RL and W_RNN_RL also used WT; however, they used both high and low frequencies, which provide the overall shape and waveform information. Information for high frequency can explain that it is possible to prepare for volatility. In other words, using high frequency as input data is significant in deep reinforcement learning research.

Tables 8 and 9 show the daily returns and cumulative returns of the S&P500 index for sets with one-year test period, respectively. As can be seen, there are solutions with higher values due to a very slight difference in the buy and

Table 10 Sharpe ratio according to the difference between the experimental group and control reward

	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	HW_LSTM_RL	W_LSTM_RL	LSTM_RL	Buy and hold
Year 1	0.4962	0.0249	0.2776	0.3336	-0.0981	-0.0540	0.3059
Year 2	-0.4228	-0.8459	-0.8515	-0.6065	-0.7960	-0.6522	-0.9450
Year 3	1.3129	1.1617	1.1699	1.2239	0.7746	0.0000	0.6715
Year 4	0.8987	0.6322	0.7401	0.8534	0.5954	0.7440	0.6715
Year 5	0.1829	0.0301	0.0304	0.0822	-0.1858	0.0000	0.0677
Year 6	0.8345	0.8155	0.8014	0.8162	0.7630	0.7844	0.9395
Year 7	2.2331	2.1838	2.1562	2.1633	2.1630	2.0314	2.2287
Year 8	1.1665	1.0753	1.0754	1.1132	1.0730	1.0731	1.0894
Year 9	0.2283	0.1304	0.1306	0.0253	0.0735	0.1103	0.0324
Year 10	1.4318	1.2216	1.2204	1.3741	1.2729	1.2727	0.8860
Year 11	2.5242	2.5221	2.5202	2.5221	2.5221	2.5168	2.5897
Year 12	-0.2941	-0.4439	-0.4317	-0.4434	-0.4835	-0.4323	-0.3433
Average	1.0519	0.9329	0.9413	0.9730	0.8568	0.8100	0.8833

hold rate of return; however, most of the values show that the value of HW_LSTM_RL suggested by us is higher. In particular, in Years 2 and 12, which were bearish markets, although they suffered losses, there was a clear difference from other years. In addition, the daily and cumulative returns also show the highest HW_LSTM_RL. In particular, the cumulative return is significantly higher in the proposed structure.

Table 10 shows the difference in Sharpe ratio according to the reward for the S&P500 index. Here, columns 2 to 4 represent the rewards of the experimental group discussed in Sec. 4.3.3, and columns 5 to 7 represent the Control group discussed in Sec. 4.3.3 (with no difference in reward). Note that only the LSTM_RLs structures (HW_LSTM_RL, W_LSTM_RL, LSTM_RL) were tested in both experiments. The highest value in the control group is shown in italics. Besides Year 11, which was a bull market, the proposed rewards (which show immediate and delayed rewards accordingly) achieved better results. In reinforcement learning, it was found that using an appropriate reward according to the learning object influenced the experimental results. In addition, even if other reward types are used, HW_LSTM_RL has the highest value.

6 Conclusions

In this paper, we have investigated how to optimize portfolio, asset allocation, and trading systems based on deep reinforcement learning. We employed high-frequency data decomposed by wavelet transform (WT) from time-series data of the S&P500, DJI, KOSPI indices as input data. Then, we learned investment transactions via deep reinforcement learning. By evaluating the Sharpe ratio, daily return, and cumulative return, we found that the proposed HW_LSTM_RL structure outperformed other similar structures and Buy and Hold values. In addition, HW_RNN_RL demonstrated the second largest value, with the Sharpe ratio, daily return, and cumulative return also demonstrated the second largest values. Thus, high-frequency data decomposed into wavelets is useful information relative to learning investment transactions. In this study, the preprocessing process provided a contribution that can be used effectively to predict investment transactions in volatile markets, e.g., diversified financial markets. In addition, the proposed HW_LSTM_RL demonstrates the importance of learning network, reward, and input data configurations. In experiments conducted with structures similar, we found that data preprocessing, learning networks, and setting up appropriate rewards in deep reinforcement learning can optimize the trading system to improve learning outcomes. In other words, the proposed HW_LSTM_RL demonstrates

importance of the composition of input data, learning networks, and rewards.

7 Future works

In the data preprocessing process, only wavelet level 2 of WT was used; however, when the wavelet level is expanded (beyond level 2), the difference between the presence or absence of data in the moving average line (MA) 5 and 10 can be compared. In addition, it is possible to conduct a comparative study of the differences between wavelets other than Haar wavelets. In this study, we investigated reinforcement learning using LSTM; however, in future, we can compare the experimental results using an effective network for time-series analysis instead of LSTM. For example, we can conduct a comparative investigation using Bi-LSTM, GRU, and Transformer, which have demonstrated effective performance in natural language processing, and BERT. In addition, if the reward is expressed using the Black-Scholes equation or Heston model, which are well known in financial research, rather than a constant function, we can study how they differ.

Nomenclature

- HW_LSTM_RL : After using only high-frequency as input data, LSTM is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- W_LSTM_RL : After using high and low frequencies as input data, LSTM is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- LSTM_RL : After using without wavelet transform as input data, LSTM is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- HW_RNN_RL : After using only high frequency as input data, an RNN is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- W_RNN_RL : After using high and low frequencies as input data, an RNN is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- RNN_RL : After using without wavelet transform as input data, an RNN is used to determine actions and reinforce learning by receiving rewards according to the states and actions.
- Buy and Hold : Buy at first start and then retain.

- {LSTM_RLs} : A word referring to HW_LSTM_RL, W_LSTM_RL, and LSTM_RL.
- {RNN_RLs} : A word referring to HW_RNN_RL, W_RNN_RL, and RNN_RL.

Acknowledgments We thank an anonymous referee for their helpful comments, which improved the clarity of the paper. This research was supported by National Research Foundation of Korea (NRF) grants funded by the Korean government (Grant No. 2015R1A5A1009350, 2018R1A2A3074566, and 2020R1I1A1A01071731). This research was also supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (Grant No. 2019R1A6A1A11051177).

References

- Addison PS (2017) The illustrated wavelet transform handbook: introductory theory and applications in science, engineering, medicine and finance. CRC press
- Bao W, Yue J, Rao Y (2017) A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PloS one* 12:e0180944
- Bengio Y, Simard P, Frasconi P et al (1994) Learning long-term dependencies with gradient descent is difficult. *IEEE Trans Neural Netw* 5(2):157–166
- Bertoluzzo F, Corazza M (2007) Making financial trading by recurrent reinforcement learning. *International Conference on Knowledge-Based and Intelligent Information and Engineering Systems*, pp 619–626
- Bertsekas DP, Bertsekas DP, Bertsekas DP, Bertsekas DP (1995) Dynamic programming and optimal control. *Athena Sci Belmont*:1(2)
- Brim A. (2020) Deep Reinforcement Learning Pairs Trading with a Double Deep Q-Network. 2020 10th Annual Computing and Communication Workshop and Conference (CCWC):0222–0227
- Carapuço J, Neves R, Horta N (2018) Reinforcement learning applied to Forex trading. *Appl Soft Comput* 73:783–794
- Carta S, Corrigan A, Ferreira A, Podda AS, Recupero DR (2020) A multi-layer and multi-ensemble stock trader using deep learning and deep reinforcement learning. *Appl Intell*:1–17
- Choe HJ, Lee J, Lee J-K (2018) Malliavin calculus for subordinated lévy process. *Chaos Solitons Fract* 116:392–401
- Dempster MA, Leemans V (2006) An automated FX trading system using adaptive reinforcement learning. *Expert Syst Appl* 30(3):543–552
- Deng Y, Bao F, Kong Y, Ren Z, Dai Q (2016) Deep direct reinforcement learning for financial signal representation and trading. *IEEE Trans Neural Netw Learn Syst* 28(3):653–664
- Deng Y, Kong Y, Bao F, Dai Q (2015) Sparse coding-inspired optimal trading system for HFT industry. *IEEE Trans Ind Inf* 11(2):467–475
- Fengqian D, Chao L (2020) An adaptive financial trading system using deep reinforcement learning with candlestick decomposing features. *IEEE Access* 8:63666–63678
- Gao Z, Gao Y, Hu Y, Jiang Z, Su J (2020) Application of Deep Q-Network in Portfolio Management. 2020 5th IEEE International Conference on Big Data Analytics (ICBDA):268–275
- Gold C (2003) FX Trading via recurrent reinforcement learning. 2003. *IEEE International Conference on Computational Intelligence for Financial Engineering*, 2003. Proceedings, pp 363–370
- Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput* 9(8):1735–1780
- Hsieh T-J, Hsiao H-F, Yeh W-C (2011) Forecasting stock markets using wavelet transforms and recurrent neural networks: an integrated system based on artificial bee colony algorithm. *Appl Soft Comput* 11(2):2510–2525
- Hu Y-J, Lin S-J (2019) Deep reinforcement learning for optimizing finance portfolio management. 2019 *Amity International Conference on Artificial Intelligence (AICAI)*. IEEE, pp 14–20
- Kingma DP, Ba J (2014) Adam: A method for stochastic optimization. [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)
- Krizhevsky A, Sutskever I, Hinton GE (2012) Imagenet classification with deep convolutional neural networks. *Adv Neural Inf Process Syst*:1097–1105
- Lahmiri S (2014) Wavelet low-and high-frequency components as features for predicting stock prices with backpropagation neural networks. *J King Saud Univ-Comput Inf Sci* 26(2):218–227
- Li J, Shi Z, Li X (2006) Genetic programming with wavelet-based indicators for financial forecasting. *Trans Inst Meas Control* 28(3):285–297
- Li Y, Ni P, Chang V (2019) Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing*:1–18
- Li Y, Ni P, Chang V (2019) An Empirical Research on the Investment Strategy of Stock Market based on Deep Reinforcement Learning model. *COMPLEXIS*:52–58
- Mallat SG (1989) A theory for multiresolution signal decomposition: the wavelet representation. *IEEE Trans Pattern Anal Mach Intell*:674–693
- Mehala N, Dahiya R (2008) A comparative study of FFT, STFT and wavelet techniques for induction machine fault diagnostic analysis. *Proceedings of the 7th WSEAS international conference on computational intelligence, man-machine systems and cybernetics*, Cairo, pp 2931
- Meng TL, Khushi M (2019) Reinforcement learning in financial markets. *Data* 4(3):110
- Moody J, Saffell M (2001) Learning to trade via direct reinforcement. *IEEE Trans Neural Netw* 12(4):875–889
- Moody J, Wu L, Liao Y, Saffell M (1998) Performance functions and reinforcement learning for trading systems and portfolios. *J Forecast* 17(5-6):441–470
- M'ng JCP, Mehralizadeh M (2016) Forecasting East Asian indices futures via a novel hybrid of wavelet-PCA denoising and artificial neural network models. *PloS one* 11(6):e0156338
- Neuneier R (1996) Optimal asset allocation using adaptive dynamic programming. *Adv Neural Inf Process Syst*:952–958
- Qiu J, Wang B, Zhou C (2020) Forecasting stock prices with long-short term memory neural network based on attention mechanism. *PloS one* 15(1):e0227222
- Renaud O, Murtagh F, Starck J-L (2002) Wavelet-based forecasting of short and long memory time series. *Université de Genève/Faculté des sciences économiques et sociales*
- Rumelhart DE, Hinton GE, Williams RJ (1985) Learning internal representations by error propagation. *California Univ San Diego La Jolla Inst for Cognitive Science*
- Sharpe WF (1994) The sharpe ratio. *J Portfolio Manag* 21(1):49–58
- Sherstov AA, Stone P (2004) Three automated stock-trading agents: a comparative study. *Int Workshop Agent-Mediated Electron Commerce*:173–187
- Soleymani F, Paquet E (2020) Financial Portfolio Optimization with Online Deep Reinforcement Learning and Restricted Stacked Autoencoder-DeepBreath. *Expert Syst Appl* 113456
- Stanković RS, Falkowski BJ (2003) The Haar wavelet transform: its status and achievements. *Comput Electr Eng* 29(1):25–44

39. Sundermeyer M, Schlüter R, Ney H (2012) LSTM neural networks for language modeling. Thirteenth annual conference of the international speech communication association
40. Sutton RS, Barto AG (2018) Reinforcement learning: An introduction. MIT press
41. Walnut DF (2013) An introduction to wavelet analysis. Springer Science & Business Media
42. Watkins CJ, Dayan P (1992) Q-learning. Mach Learn 8(3-4):279–292
43. Werbos PJ et al (1960) Backpropagation through time: what it does and how to do it. Proc IEEE 78(10):1550–1560
44. Widrow B, Hoff ME (1960) Adaptive switching circuits. Stanford Univ Ca Stanford Electronics Lab
45. wikibook (2018) <https://github.com/quantylab/rltrader>
46. Wu X, Chen H, Wang J, Troiano L, Loia V, Fujita H (2020) Adaptive Stock Trading Strategies with Deep Reinforcement Learning Methods. Information Sciences
47. Xiong Z, Liu X-Y, Zhong S, Yang H, Walid A (2018) Practical deep reinforcement learning approach for stock trading. arXiv:1811.07522
48. Zhang B.-L., Coggins R, Jabri MA, Dersch D, Flower B (2001) Multiresolution forecasting for futures trading using wavelet decompositions. IEEE Trans Neural Netw 12(4):765–775

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.